Layered Testbench for Verification

Rehan Naeem

University of Engineering and Technology Lahore

September 2025

Outline

Clocked Adder

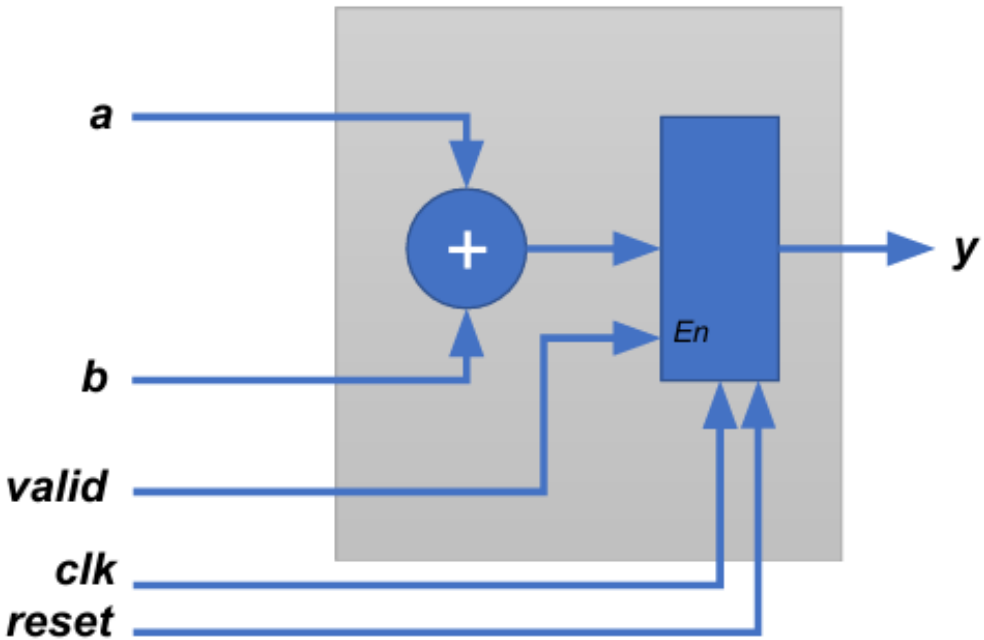


Figure: Single-bit Adder

Layered Testbench

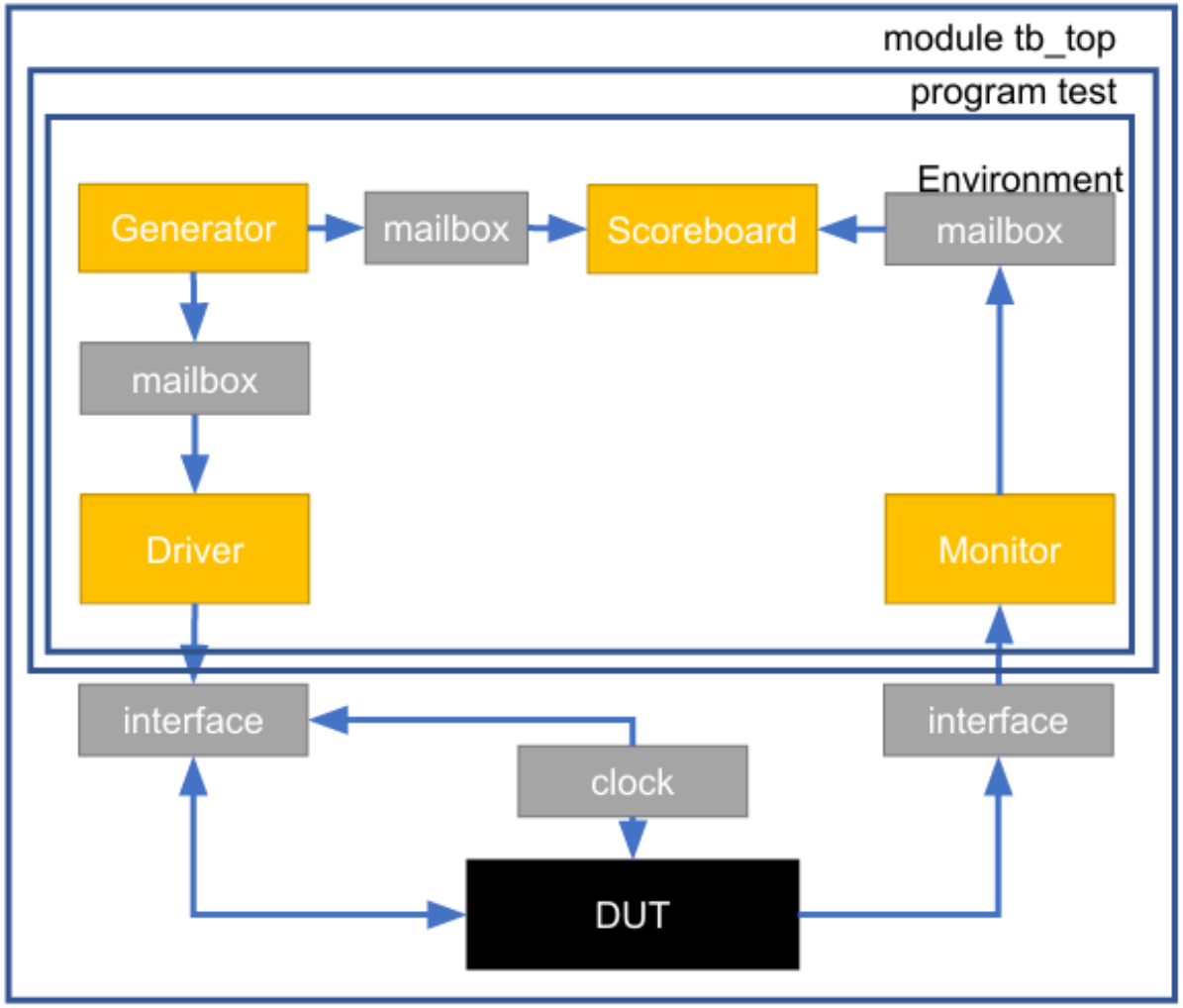


Figure: Transactors

Components of Layered Testbench

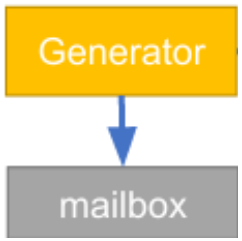


Figure: Transactors

Components of Layered Testbench

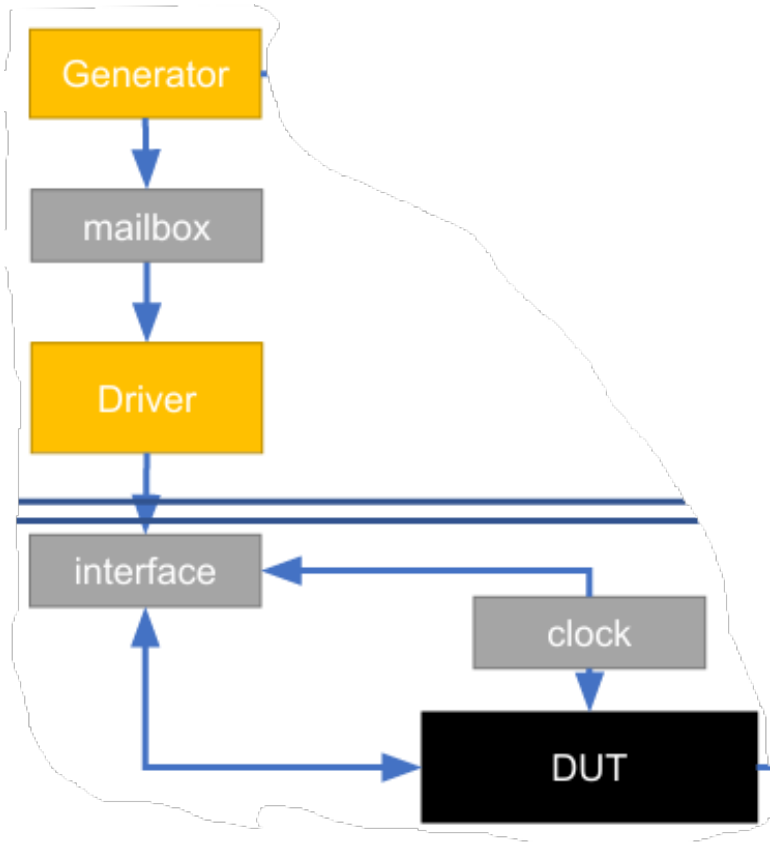


Figure: Transactors

Components of Layered Testbench

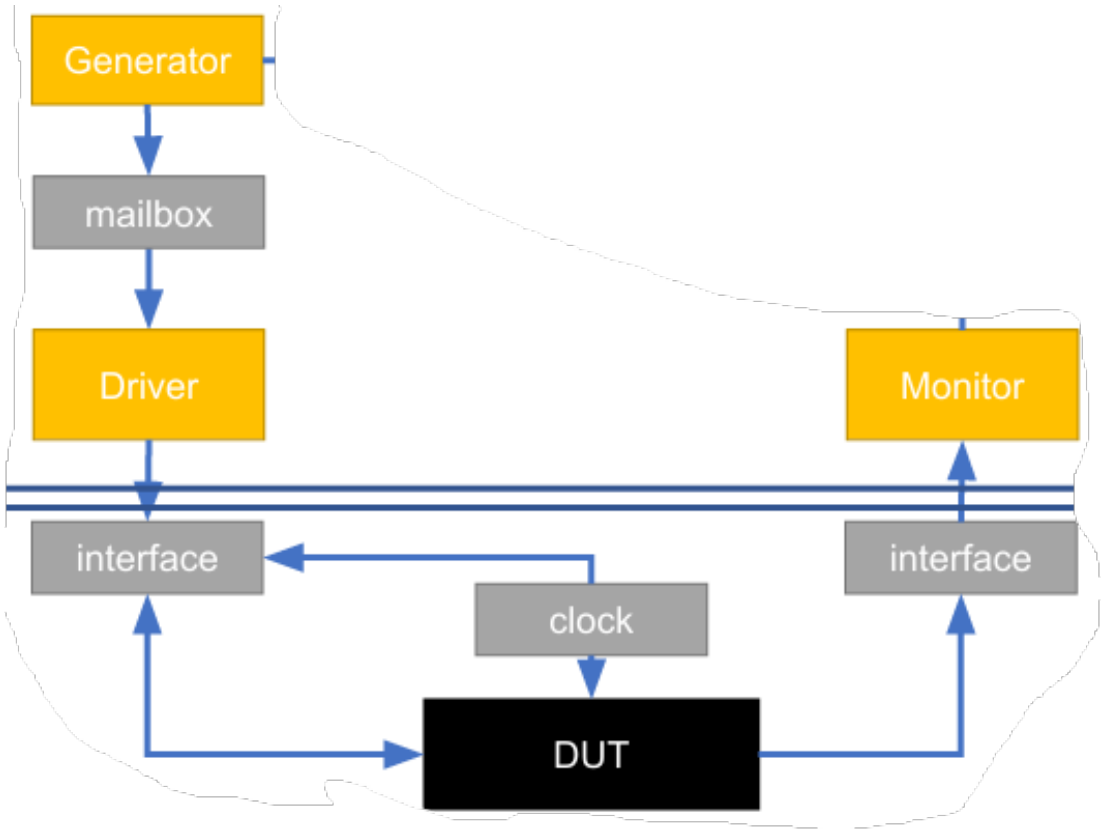


Figure: Transactors

Components of Layered Testbench

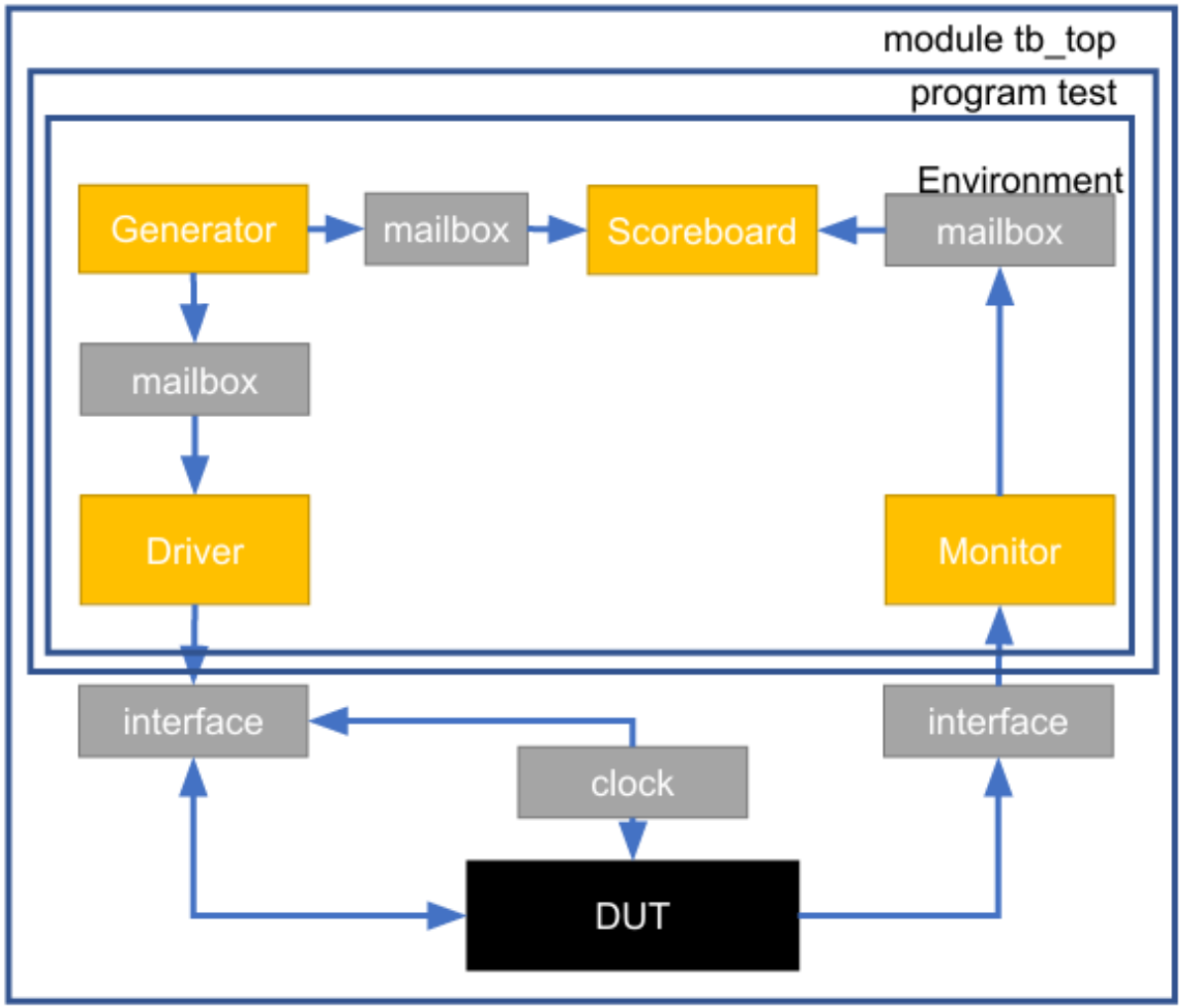
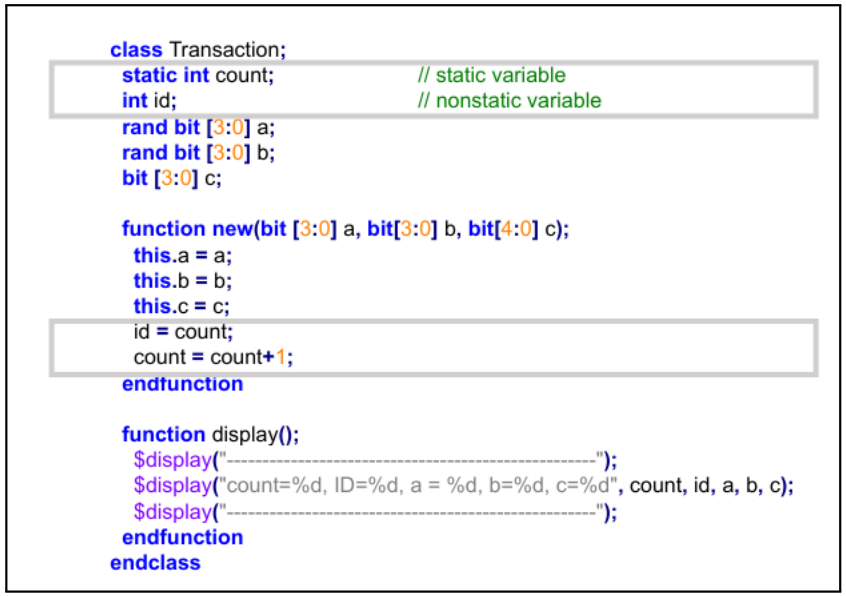


Figure: Transactors

Generator

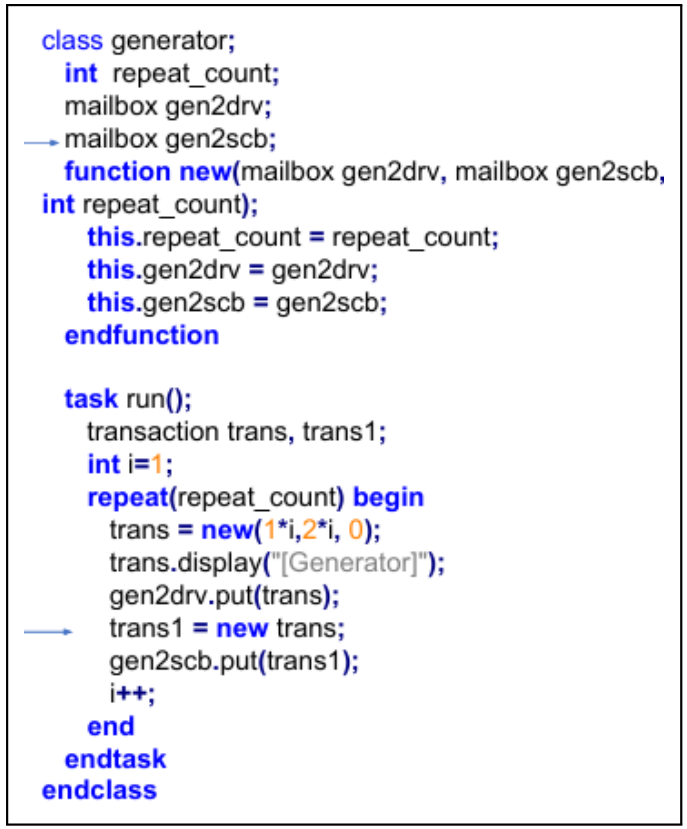


```
class Transaction;
  static int count;                    // static variable
  int id;                              // nonstatic variable
  rand bit [3:0] a;
  rand bit [3:0] b;
  bit [3:0] c;

  function new(bit [3:0] a, bit[3:0] b, bit[4:0] c);
    this.a = a;
    this.b = b;
    this.c = c;
    id = count;
    count = count+1;
  endfunction

  function display();
    $display("-----------------------------------------------------");
    $display("count=%d, ID=%d, a = %d, b=%d, c=%d", count, id, a, b, c);
    $display("-----------------------------------------------------");
  endfunction
endclass
```

(a) Transactor Code



```
class generator;
  int  repeat_count;
  mailbox gen2drv;
  mailbox gen2scb;
  function new(mailbox gen2drv, mailbox gen2scb,
int repeat_count);
    this.repeat_count = repeat_count;
    this.gen2drv = gen2drv;
    this.gen2scb = gen2scb;
  endfunction

  task run();
    transaction trans, trans1;
    int i=1;
    repeat(repeat_count) begin
      trans = new(1*i,2*i, 0);
      trans.display("[Generator]");
      gen2drv.put(trans);
      trans1 = new trans;
      gen2scb.put(trans1);
      i++;
    end
  endtask
endclass
```

(b) Generator Code

Driver


```
class driver;
     mailbox gen2driv;
     int repeat count;
     virtual ra intf.drv rinf;
     function new(mailbox gen2driv, int repeat_count, virtual ra_intf.drv rinf);
          this.gen2driv = gen2driv;
          this.repeat count = repeat_count;
          this.rinf = rinf;
     endfunction

     task run();
          transaction trans;
          repeat(repeat count) begin
               gen2driv.get(trans);
               trans.display("[DRIVER]");
               reset sequence();
               drive_signals(trans.a, trans.b);
          end
     endtask

     task drive signals(input logic [3:0] a, b);
          @(posedge rinf.clk); rinf.a <= a; rinf.b <= b; rinf.valid <= 1;
          @(posedge rinf.clk); rinf.valid <= 0;
          @(posedge rinf.clk);
     endtask
endclass
```

Figure: Driver Code

Monitor


```
class monitor;
   mailbox mon2scb;
   int repeat count;
   virtual ra intf.mon rinf;
   function new(mailbox mon2scb, int repeat_count,
virtual ra_intf.mon rinf);
      this.mon2scb = mon2scb;
      this.repeat count = repeat_count;
      this.rinf = rinf;
   endfunction

   task run();
      transaction trans;
      int ncount = 0;
      while(ncount<repeat count) begin
         @(posedge rinf.clk);
         if(!rinf.reset && rinf.valid) begin
            @(posedge rinf.clk);
            trans = new(rinf.a, rinf.b, rinf.c);
            mon2scb.put(trans);
            trans.display("[Monitor]");
            ncount++;
         end
      end
   endtask
endclass
```

Figure: Monitor Code

Scoreboard


```
class scoreboard;
  mailbox mon2scb, gen2scb;
  int repeat_count;

  function new(mailbox gen2scb, mailbox mon2scb,
       int repeat_count);
    this.mon2scb = mon2scb;
    this.gen2scb = gen2scb;
    this.repeat_count = repeat_count;
  endfunction

  task run();
    transaction t_fromdrv, t_frommon;
    int ncount = 0, nsuccess =0, nfails = 0;
    while(ncount<repeat_count) begin
      gen2scb.get(t_fromdrv);
      mon2scb.get(t_frommon);
      if ((t_fromdrv.a+t_fromdrv.b)!=t_frommon.c) nfails++;
      else nsuccess++;
      ncount++;
    end
    $display("Total Success = %0d, Total Failures = %0d",
             nsuccess, nfails);
  endtask
endclass
```

Figure: Scoreboard Code

Environment

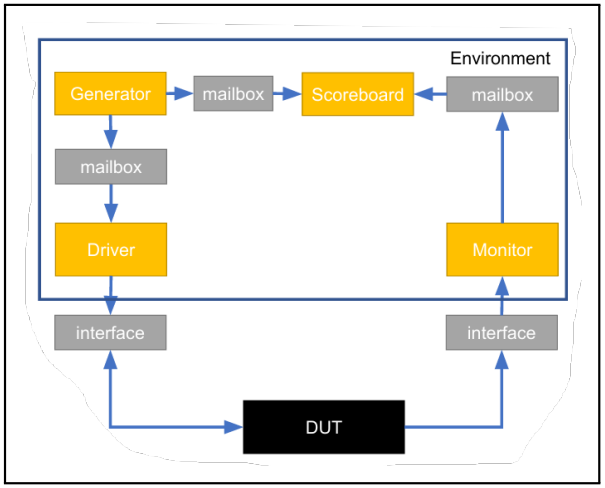

```
`include "transaction.sv"
`include "generator.sv"
`include "driver.sv"
`include "monitor.sv"
`include "scoreboard.sv"
class environment;
  mailbox gen2drv, mon2scb,  gen2scb;

  generator gen;
  driver drv;
  monitor mon;
  scoreboard scb;

  function new(virtual ra_intf rinf, int repeat_count);
    this.gen2drv = new();
    this.mon2scb = new();
    this.gen2scb = new();
    this.gen = new(gen2drv, gen2scb, repeat_count);
    this.drv = new(gen2drv, repeat_count, rinf);
    this.mon = new(mon2scb, repeat_count, rinf);
    this.scb = new(gen2scb, mon2scb, repeat_count);
  endfunction

  task run();
    fork
      gen.run();      drv.run();      mon.run();      scb.run();
    join
  endtask;
endclass
```

(a) Environment Code



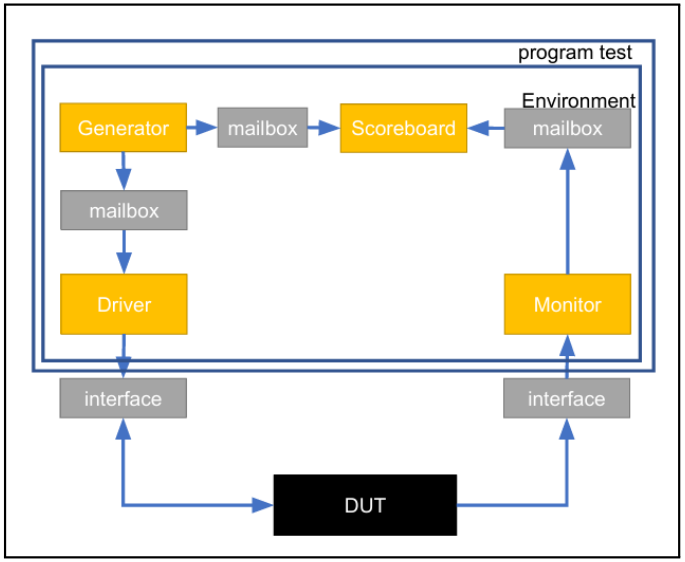
(b) Block Diagram

Testbench


```
`include "environment.sv"
program automatic test(ra_intf rinf);

  initial begin
    environment env;
    env = new(rinf, 10);
    env.run();
    $stop;
  end
endprogram
```

(a) Environment Code



(b) Block Diagram

Top Module

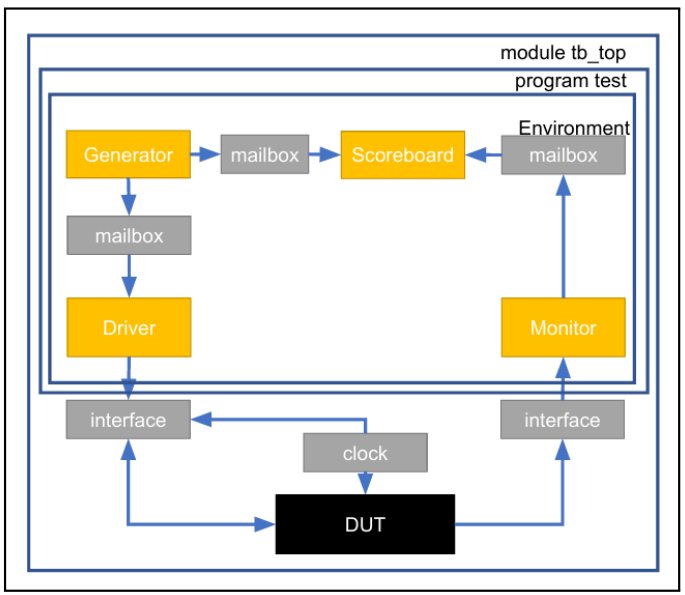

```
`include "ra_intf.sv"
module tb_top;

        // Clock generator
        bit clk;
        initial begin
                clk = 0;
                forever #20 clk = ~clk;
        end

        ra_intf rinf(clk);
        radder ra(rinf);
        test ta(rinf);

endmodule
```

(a) Environment Code



(b) Block Diagram

References

Dr Ubaid Ullah Fayyaz.
System verilog for verification (slides).

Spears.
System Verilog for Verification.